
Valkyrie Package Documentation, version 0.3.0

James R. Swift, lordkekz, eltos, tinger <tinger@tinger.dev>, jneug, TimeTravelPenguin and NanamiNakano
Typst Community

July 23, 2026

Contents

1. Preface	4
1.1. Terminology	4
1.2. Specific language	4
1.3. Use cases	4
2. Examples	4
3. Data structures	4
3.1. Schema	4
4. API Reference	4
4.1. alignment	4
4.2. angle	4
4.3. any	5
4.4. array	5
4.5. boolean	5
4.6. bytes	5
4.7. choice	5
4.8. color	5
4.9. content	6
4.10. date	6
4.11. dictionary	6
4.12. direction	6
4.13. either	6
4.14. fraction	7
4.15. function	7
4.16. gradient	7
4.17. label	7
4.18. length	7
4.19. location	7
4.20. parse	8
4.21. plugin	8
4.22. ratio	8
4.23. regex	8
4.24. relative	8
4.25. selector	9
4.26. string	9
4.27. stroke	9
4.28. symbol	9
4.29. tuple	9
4.30. version	10
5. API Reference (Predefined schemas)	10
5.1. author	10
6. API Reference (Assertions)	10
6.1. contains	10
6.2. ends-with	10
6.3. matches	10
6.4. one-of	10
6.5. starts-with	10
6.6. length	11

6.6.1. equals	11
6.6.2. max	11
6.6.3. min	11
6.6.4. neq	11
7. API Reference (Coercions)	11
7.1. array	11
7.2. content	12
7.3. date	12
7.4. dictionary	12
8. API Reference (Advanced)	13
8.1. base-type	13
8.2. z-ctx	13

1. Preface

1.1. Terminology

As this package introduces several type-like objects, the Tidy style has had these added for clarity. At present, these are `schema` (to represent type-validating objects), `z-ctx` (to represent the current state of the parsing heuristic), and `scope` (an array of strings that represents the parent object of values being parsed). `internal` represents arguments that, while settable by the end-user, should be reserved for internal or advanced usage.

In general, users of this package will only need to be aware of the `schema` type (see Section 3.1)

1.2. Specific language

The key words **MUST**, **MUST NOT**, **REQUIRED**, **SHALL**, **SHALL-NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in RFC 2119.

1.3. Use cases

The interface for a template that a user expects and that the developer has implemented are rarely one and the same. Instead, the user will apply common sense and the developer will put in somewhere between a token- and a whole-hearted- attempt at making their interface intuitive. Contrary to what one might expect, this makes it more difficult for the end user to correctly guess the interface as different developers will disagree on what is and isn't intuitive, and what edge cases the developer is willing to cover.

By first providing a low-level set of tools for validating primitives upon which more complicated schemas can be defined, Valkyrie handles both the micro and macro of input validation.

2. Examples

3. Data structures

3.1. Schema

4. API Reference

4.1. alignment

see `#z.base-type()`

4.1.1. Parameters

`alignment()` -> `schema`

4.2. angle

see `#z.base-type()`

4.2.1. Parameters

`angle()` -> `schema`

4.3. any

see `#z.base-type()`

4.3.1. Parameters

`any()` -> `schema`

4.4. array

4.4.1. Parameters

```
array(  
  name,  
  assertions,  
  min,  
  max,  
  ..args  
)
```

4.5. boolean

see `#z.base-type()`

4.5.1. Parameters

`boolean()` -> `schema`

4.6. bytes

see `#z.base-type()`

4.6.1. Parameters

`bytes()` -> `schema`

4.7. choice

see `#z.base-type()`

4.7.1. Parameters

```
choice(  
  list,  
  assertions,  
  ..args  
) -> schema
```

4.8. color

see `#z.base-type()`

4.8.1. Parameters

`color()` -> `schema`

4.9. content

see `#z.base-type()`

4.9.1. Parameters

`content()` -> `schema`

4.10. date

see `#z.base-type()`

4.10.1. Parameters

`date()` -> `schema`

4.11. dictionary

Valkyrie schema generator for dictionary types. Named arguments define validation schema for entries. Dictionaries can be nested.

4.11.1. Parameters

```
dictionary(  
  dictionary-schema,  
  name,  
  default,  
  pre-transform,  
  aliases,  
  ..args  
) -> schema
```

4.12. direction

see `#z.base-type()`

4.12.1. Parameters

`direction()` -> `schema`

4.13. either

Valkyrie schema generator for objects that can be any of multiple types.

4.13.1. Parameters

```
either(  
  strict,  
  ..args  
) -> schema
```

4.14. fraction

see [#z.base-type\(\)](#)

4.14.1. Parameters

```
fraction() -> schema
```

4.15. function

see [#z.base-type\(\)](#)

4.15.1. Parameters

```
function() -> schema
```

4.16. gradient

see [#z.base-type\(\)](#)

4.16.1. Parameters

```
gradient() -> schema
```

4.17. label

see [#z.base-type\(\)](#)

4.17.1. Parameters

```
label() -> schema
```

4.18. length

see [#z.base-type\(\)](#)

4.18.1. Parameters

```
length() -> schema
```

4.19. location

see [#z.base-type\(\)](#)

4.19.1. Parameters

`location()` -> `schema`

4.20. parse

Interrogate an object with respect to a schema.

4.20.1. Parameters

```
parse(  
  object: any ,  
  schemas: schema ,  
  ctx: ctx ,  
  scope: array  
)
```

4.20.1.1. object any

Object to interrogate.

4.21. plugin

see `#z.base-type()`

4.21.1. Parameters

`plugin()` -> `schema`

4.22. ratio

see `#z.base-type()`

4.22.1. Parameters

`ratio()` -> `schema`

4.23. regex

see `#z.base-type()`

4.23.1. Parameters

`regex()` -> `schema`

4.24. relative

see `#z.base-type()`

4.24.1. Parameters

`relative()` -> `schema`

4.25. selector

see `#z.base-type()`

4.25.1. Parameters

`selector()` -> `schema`

4.26. string

Valkyrie schema generator for strings

4.26.1. Parameters

```
string(  
    assertions,  
    min,  
    max,  
    ..args  
) -> schema
```

4.27. stroke

see `#z.base-type()`

4.27.1. Parameters

`stroke()` -> `schema`

4.28. symbol

see `#z.base-type()`

4.28.1. Parameters

`symbol()` -> `schema`

4.29. tuple

Valkyrie schema generator for an array type with positional type requiements. If all entries have the same type, see `#z.array()` exact (bool): Requires a tuple to match in length

4.29.1. Parameters

```
tuple(  
    exact,  
    ..args  
) -> schema
```

4.30. version

see `#z.base-type()`

4.30.1. Parameters

`version()` -> `schema`

5. API Reference (Predefined schemas)

5.1. author `schema`

6. API Reference (Assertions)

6.1. contains

Asserts that a tested value contains the argument (string)

6.1.1. Parameters

`contains(value)`

6.2. ends-with

Asserts that a tested value ends with the argument (string)

6.2.1. Parameters

`ends-with(value)`

6.3. matches

Asserts that a tested value matches with the needle argument (string)

6.3.1. Parameters

```
matches(  
  needle,  
  message  
)
```

6.4. one-of

Asserts that the given value is contained within the provided list. Useful for complicated enumeration types.

- list (array): An array of inputs considered valid.

6.4.1. Parameters

`one-of(list)`

6.5. starts-with

Asserts that a tested value starts with the argument (string)

6.5.1. Parameters

`starts-with(value)`

6.6. length

6.6.1. equals

Asserts that tested value's length is exactly equal to argument

6.6.1.1. Parameters

`equals(arg)`

6.6.2. max

Asserts that tested value's length is less than or equal to argument

6.6.2.1. Parameters

`max(rhs)`

6.6.3. min

Asserts that tested value's length is greater than or equal to argument

6.6.3.1. Parameters

`min(rhs)`

6.6.4. neq

Asserts that tested value's length is not equal to argument

6.6.4.1. Parameters

`neq(arg)`

7. API Reference (Coercions)

7.1. array

If the tested value is not already of array type, it is transformed into an array of size 1

```
#let schema = z.array(  
  pre-transform: z.coerce.array,  
  z.string()  
)  
  
#z.parse("Hello", schema) \  
#z.parse(("Hello", "world"), schema)
```

```
("Hello",)  
("Hello", "world")
```

7.1.1. Parameters

```
array(  
  self,  
  it  
)
```

7.2. content

Tested value is forcibly converted to content type

```
#let schema = z.content(  
  pre-transform: z.coerce.content  
)  
  
#type(z.parse("Hello", schema)) \  
#type(z.parse(123456, schema))
```

```
content  
content
```

7.2.1. Parameters

```
content(  
  self,  
  it  
)
```

7.3. date

An attempt is made to convert string, numeric, or dictionary inputs into datetime objects

```
#let schema = z.date(  
  pre-transform: z.coerce.date  
)  
  
#z.parse(2020, schema) \  
#z.parse("2020-03-15", schema) \  
#z.parse("2020/03/15", schema) \  
#z.parse((year: 2020, month: 3, day: 15),  
schema) \  

```

```
datetime(year: 2020, month: 1, day: 1)  
datetime(year: 2020, month: 3, day: 15)  
datetime(year: 2020, month: 3, day: 15)  
datetime(year: 2020, month: 3, day: 15)
```

7.3.1. Parameters

```
date(  
  self,  
  it  
)
```

7.4. dictionary

If the tested value is not already of dictionary type, the function provided as argument is expected to return a dictionary type with a shape that passes validation.

```
#let schema = z.dictionary(  
  pre-transform:  
    z.coerce.dictionary((it)=>(name: it)),  
  (name: z.string())  
)  
  
#z.parse("Hello", schema) \  
#z.parse((name: "Hello"), schema)
```

```
(name: "Hello")  
(name: "Hello")
```

- fn (function): Transformation function that the tested value and returns a dictionary that has a shape that passes validation.

7.4.1. Parameters

`dictionary(fn)`

8. API Reference (Advanced)

8.1. base-type

Schema generator. Provides default values for when defining custom types.

8.1.1. Parameters

```
base-type(  
  name,  
  description,  
  optional,  
  default,  
  types,  
  assertions,  
  pre-transform,  
  post-transform  
)
```

8.2. z-ctx

This is a utility function for setting contextual flags that are used during validation of objects against schemas. Currently, the following flags are described within the API:

strict If set, this flag adds the requirement that there are no entries in dictionary types that are not described by the validation schema.

soft-error If set, this flag silences errors from failed validation parses. It is used internally for types that should not error on validation failures. See `#z.either()`

remove-optional-none This flag is used to indicate that, when parsing a dictionary, if a field is marked as optional in the schema, and its value is `none`, the key should be absent from the validated dictionary, otherwise, that it **SHOULD** be present in the validated dictionary with a value of `none`

8.2.1. Parameters

```
z-ctx(  
  parent: dictionary default,  
  ..flags: arguments  
) -> z-ctx
```

8.2.1.1. parent dictionary or default

The parent context from which to derive. The parent context provides the default value of any missing flags

Default: `(:)`

8.2.1.2. ..flags arguments

Variadic contextual flags to set. Positional arguments are discarded.